



since we are dealing with the lower level APIs ,we have to work at the sparkcontext level which is available under the sparksession

first thing to execute code on spark cluster is first we need to create spark session

```
In [1]: from pyspark.sql import SparkSession
import getpass
username = getpass.getuser()
spark = SparkSession. \
builder. \
config('spark.ui.port', '0'). \
config("spark.sql.warehouse.dir", f"/user/{username}/warehouse"). \
enableHiveSupport(). \
master('yarn'). \
getOrCreate()
```

Loading Data into RDD :

```
In [2]: orders_rdd = spark.sparkContext.textFile("/public/trendytech/retail_db/orders/
```

```
In [ ]: # to check the content now just we need to give the
orders_rdd.collect() # this will display all the contents which is stored in
or
just give here
orders_rdd.take(10) # take is an action here and it prints the first 10 records
```

1. As per the customer requirements we need to find out the how many orders under each category
2. For ex- how many orders got placed under the closed category , complete category :

we have data like this : '1,2013-07-25 00:00:00.0,11599,CLOSED', '2,2013-07-25 00:00:00.0,256,PENDING PAYMENT', '3,2013-07-25 00:00:00.0,12111,COMPLETE', '4,2013-07-25 00:00:00.0,8827,CLOSED', '5,2013-07-25 00:00:00.0,11318,COMPLETE' we want output looks like this at this stage: [('CLOSED', 1), ('PENDING PAYMENT', 1), ('COMPLETE', 1), ('CLOSED', 1), ('COMPLETE', 1), ('COMPLETE', 1), ('COMPLETE', 1), ('PROCESSING', 1), ('PENDING_PAYMENT', 1), ('PENDING_PAYMENT', 1)]# (every_status,1) - for this it is like one-to-one mapping between the input and output # we need to take order_status and add 1 there as output to it. ## so we need to perform the split operation and then perform the mapping here

```
In [3]: # considering each row as x :  
# x.split() gives us a list and we need 3rd index  
# since rdd's are immutable ,we need to create a new rdd here to store the re  
mapped_rdd = orders_rdd.map(lambda x:(x.split(",")[3],1))
```

```
In [4]: mapped_rdd.take(10)  
# when we perform this action step, then we get the result as :
```

```
Out[4]: [('CLOSED', 1),  
        ('PENDING_PAYMENT', 1),  
        ('COMPLETE', 1),  
        ('CLOSED', 1),  
        ('COMPLETE', 1),  
        ('COMPLETE', 1),  
        ('COMPLETE', 1),  
        ('PROCESSING', 1),  
        ('PENDING_PAYMENT', 1),  
        ('PENDING_PAYMENT', 1)]
```

now we want to do the aggregation based on the keys here # so give below command here : # so we need to perform reduce based on keys

```
In [5]: reduced_rdd = mapped_rdd.reduceByKey(lambda x,y:x+y)  
# this is transformation and is lazy and executes when an action is performed
```

```
In [6]: reduced_rdd.collect() # when it run then action will be performed and we get t
```

```
Out[6]: [('CLOSED', 7556),  
        ('CANCELED', 1428),  
        ('COMPLETE', 22899),  
        ('PENDING_PAYMENT', 15030),  
        ('SUSPECTED_FRAUD', 1558),  
        ('PENDING', 7610),  
        ('ON_HOLD', 3798),  
        ('PROCESSING', 8275),  
        ('PAYMENT_REVIEW', 729)]
```

```
In [7]: # when we want to arrange the orders in the increasing order , then use sortE  
reduced_sorted = reduced_rdd.sortBy(lambda x:x[1],False)
```

```
In [8]: reduced_sorted.collect() # after giving this collect command on that variabl
```

```
Out[8]: [('COMPLETE', 22899),  
        ('PENDING_PAYMENT', 15030),  
        ('PROCESSING', 8275),  
        ('PENDING', 7610),  
        ('CLOSED', 7556),  
        ('ON_HOLD', 3798),  
        ('SUSPECTED_FRAUD', 1558),  
        ('CANCELED', 1428),  
        ('PAYMENT_REVIEW', 729)]
```

find the premium customers (Top 10 who placed the most number of orders)

so we need to take help of customer_id's here and add them

```
In [9]: # find the premium customers (Top 10 who placed the most number of orders)
# example: one input row --> one output row .. so 1000 input rows ---> 1000 c
customers_mapped = orders_rdd.map(lambda x:(x.split(",")[2],1))
```

```
In [10]: customers_mapped.take(5)
```

```
Out[10]: [('11599', 1), ('256', 1), ('12111', 1), ('8827', 1), ('11318', 1)]
```

```
In [11]: customers_aggregated = customers_mapped.reduceByKey(lambda x,y:x+y)
```

```
In [12]: customers_aggregated.take(20)
```

```
Out[12]: [('256', 10),
          ('12111', 6),
          ('11318', 6),
          ('7130', 7),
          ('2911', 6),
          ('5657', 12),
          ('9149', 4),
          ('9842', 7),
          ('7276', 5),
          ('9488', 7),
          ('2711', 3),
          ('333', 6),
          ('656', 5),
          ('6983', 6),
          ('4189', 3),
          ('4840', 2),
          ('5863', 6),
          ('8214', 5),
          ('7776', 8),
          ('1549', 4)]
```

```
In [13]: # sort by above result based on column 2 which is x[1]
customers_sorted = customers_aggregated.sortBy(lambda x:x[1],False)
```

```
In [14]: customers_sorted.take(10)
# this is the result what we want
```

```
Out[14]: [('5897', 16),
          ('6316', 16),
          ('12431', 16),
          ('569', 16),
          ('4320', 15),
          ('221', 15),
          ('5624', 15),
          ('5283', 15),
          ('12284', 15),
          ('5654', 15)]
```

distinct count of customers who placed atleast one order

```
In [15]: # we need only the customer id
distinct_customers = orders_rdd.map(lambda x:(x.split(",")[2])).distinct()
```

```
In [16]: distinct_customers.count() # here count is an action sometimes it will be tran
```

```
Out[16]: 12405
```

```
In [17]: orders_rdd.count()
```

```
Out[17]: 68883
```

which customers has the maximum number of CLOSED orders

```
In [18]: filtered_orders = orders_rdd.filter(lambda x:x.split(",")[3] == 'CLOSED')
```

```
In [19]: filtered_orders.take(20)
```

```
Out[19]: ['1,2013-07-25 00:00:00.0,11599,CLOSED',
          '4,2013-07-25 00:00:00.0,8827,CLOSED',
          '12,2013-07-25 00:00:00.0,1837,CLOSED',
          '18,2013-07-25 00:00:00.0,1205,CLOSED',
          '24,2013-07-25 00:00:00.0,11441,CLOSED',
          '25,2013-07-25 00:00:00.0,9503,CLOSED',
          '37,2013-07-25 00:00:00.0,5863,CLOSED',
          '51,2013-07-25 00:00:00.0,12271,CLOSED',
          '57,2013-07-25 00:00:00.0,7073,CLOSED',
          '61,2013-07-25 00:00:00.0,4791,CLOSED',
          '62,2013-07-25 00:00:00.0,9111,CLOSED',
          '87,2013-07-25 00:00:00.0,3065,CLOSED',
          '90,2013-07-25 00:00:00.0,9131,CLOSED',
          '101,2013-07-25 00:00:00.0,5116,CLOSED',
          '116,2013-07-26 00:00:00.0,8763,CLOSED',
          '129,2013-07-26 00:00:00.0,9937,CLOSED',
          '133,2013-07-26 00:00:00.0,10604,CLOSED',
          '191,2013-07-26 00:00:00.0,16,CLOSED',
          '201,2013-07-26 00:00:00.0,9055,CLOSED',
          '211,2013-07-26 00:00:00.0,10372,CLOSED']
```

```
In [20]: # we need the customer id here
         filtered_mapped = filtered_orders.map(lambda x:(x.split(",")[2],1))
```

```
In [21]: filtered_mapped.take(10)
```

```
Out[21]: [('11599', 1),
          ('8827', 1),
          ('1837', 1),
          ('1205', 1),
          ('11441', 1),
          ('9503', 1),
          ('5863', 1),
          ('12271', 1),
          ('7073', 1),
          ('4791', 1)]
```

```
In [22]: filtered_aggregated = filtered_mapped.reduceByKey(lambda x,y:x+y)
```

```
In [23]: filtered_aggregated.take(20)
```

```
Out[23]: [('5863', 1),
          ('12271', 2),
          ('7073', 1),
          ('3065', 2),
          ('5116', 2),
          ('8763', 1),
          ('10604', 2),
          ('16', 1),
          ('9055', 3),
          ('10372', 3),
          ('11715', 1),
          ('5925', 1),
          ('8309', 3),
          ('948', 1),
          ('5191', 1),
          ('7650', 2),
          ('4199', 2),
          ('6989', 1),
          ('5011', 4),
          ('11394', 1)]
```

```
In [24]: filtered_sorted = filtered_aggregated.sortBy(lambda x:x[1],False)
```

```
In [25]: filtered_sorted.take(10)
```

```
Out[25]: [('1833', 6),
          ('1363', 5),
          ('1687', 5),
          ('5493', 5),
          ('5011', 4),
          ('8974', 4),
          ('2321', 4),
          ('3736', 4),
          ('8368', 4),
          ('2236', 4)]
```

```
In [ ]:
```